BSc (Hons) in **Information Technology Specialized in AI**

**Faculty of Computing**

# IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

## Practical 04

> **Objective & Lecture Mapping:** In previous labs, you explored Uninformed Search strategies like BFS and UCS inside `agent.py` [cite: 1]. Today, we transition to **Informed Search**. You will enhance your agent by implementing the **A* Search** algorithm, using **Heuristics** to drastically reduce the number of explored nodes in the `visual_grid_game.py` environment[cite: 4]. You will start with basic heuristic functions and connect them to your search logic.

## Part 1: Practical Implementation — Building the A* Agent

### Step 1.1: Implementing the Heuristic Functions

> Informed search algorithms require a heuristic function, h(n), which estimates the cheapest path from the current node to the goal. You will calculate distance metrics on a 2D grid.

1. Create a new Branch called Week_04 in your Forked Github Repo.
2. Open `agent.py` and locate your `SearchAgent` class[cite: 1].
3. Add a new method named `def manhattan_distance(self, pos, goal):` that calculates the distance using the formula h(n) = |x_1 - x_2| + |y_1 - y_2| and returns the integer value.
4. Add a second method named `def euclidean_distance(self, pos, goal):` that calculates the straight-line distance using the formula h(n) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}. You will need to `import math` for this.
5. *Testing Checkpoint:* Print the outputs of both functions for a mock start position (0, 0) and goal (3, 4) to verify that Manhattan returns 7 and Euclidean returns 5.0.

### Step 1.2: Implementing A* Search

> A* evaluates nodes by combining the path cost so far, g(n), and the estimated cost to the goal, h(n). The evaluation function is f(n) = g(n) + h(n).

1. Inside `SearchAgent`, create a new method: `def astar_search(self, start_pos, goal_pos, walls, grid_size, heuristic_type='manhattan')`.
2. Initialize an empty priority queue using `heapq` and an empty set for `reached_states`.
3. Push the initial state into the priority queue. Unlike UCS which only uses g(n), your A* tuple must be formatted as: `(f_cost, g_cost, current_pos, path_taken)`. For the starting node, g(n) = 0. Calculate h(n) using your chosen heuristic method, and set f(n) = g(n) + h(n).
4. Create your standard `while` loop to process the queue. Pop the node with the lowest `f_cost`. If `current_pos == goal_pos`, return the `path_taken`. Otherwise, add `current_pos` to `reached_states`.
5. For node expansion, check the four adjacent cells (Up, Down, Left, Right). For each valid neighbor (not a wall, within bounds, and not reached): Calculate g_{new} = g_{current} + 1, h_{new} using your heuristic, and f_{new} = g_{new} + h_{new}. Push the new tuple to the priority queue.

## Step 1.3: Integrating A* into the Agent's Decision Loop

> Finally, you need to connect your new search algorithm to the agent's perception and action loop so it can run in the visual environment.

1. Navigate to the `sense_and_act(self, percept)` method in your `SearchAgent`.
2. Add an `elif self.active_algo == 'AStar':` block to handle the new algorithm alongside your existing BFS/DFS/UCS.
3. Extract the necessary global state from the percept dictionary (e.g., `percept['remaining_food']`).
4. Find the closest food item to act as the `goal_pos`. Call your `astar_search` method and store the returned list of actions in `self.plan`.
5. Open `visual_grid_game.py` and modify the `GridGameGUI` initialization to inject your `SearchAgent()` instead of the `ModelBasedAgent()`. Run the simulation and observe the optimized pathfinding!

## Part 2: Theoretical Evaluation

> Based on your implementation and Lecture 05, complete the following analytical questions.

**1. (Understand) What is the key difference between how Uniform-Cost Search (UCS) and A* Search prioritize which node to explore next?**

UCS only looks at how much it cost to reach a node so far, g(n). It has no idea where the goal is, so it explores in every direction equally.

A* looks at g(n) **plus** a heuristic guess of how far the goal still is, h(n). This means A* prefers nodes that seem closer to the goal, so it explores fewer useless nodes than UCS.

**2. (Analyze) In Step 1.1, you used Manhattan Distance. Why is Manhattan Distance considered an "admissible" heuristic for this specific 4-way movement grid, and what would happen to your A* algorithm if the heuristic was NOT admissible?**

On this grid, every move costs 1 and only changes x or y by 1. So the smallest possible number of moves between two cells (ignoring walls) is exactly |x1-x2| + |y1-y2| — which is the Manhattan distance. Walls can only make the real path longer, never shorter, so this heuristic never overestimates the true cost. That's what "admissible" means.

If the heuristic overestimated the cost, A* could get tricked into thinking a longer path is actually shorter. It might then miss the real shortest path and return a worse one. A* would still finish, but it would no longer guarantee the optimal path.

**3. (Evaluate) If we modified `visual_grid_game.py` to allow the agent to move diagonally (8-way movement), would Manhattan distance still be an admissible heuristic? Why or why not? Which metric should you switch to?**

No. With diagonal moves, the agent can cover an x-step and a y-step in one move. So the real shortest path can be shorter than what Manhattan

distance predicts - meaning it now overestimates, which breaks admissibility.

You should switch to:

- **Chebyshev distance** = max(|x1-x2|, |y1-y2|) — if a diagonal move still costs 1.
- **Euclidean distance** = √((x1-x2)² + (y1-y2)²) — if a diagonal move costs more (like √2).

**4. (Create) When targeting multiple food items simultaneously, calculating the distance to just the single closest food item is a weak heuristic. Propose (in text) a stronger heuristic for navigating the grid to eat ALL remaining food efficiently.**

instead of just measuring distance to the nearest food, use something that accounts for **all** the food left. A good option: build a Minimum Spanning Tree (MST) connecting the agent's position and every remaining food item, and use the total length of that tree as h(n).

This works better because it doesn't just chase the closest pellet — it estimates the cost of visiting all of them together, so the agent picks paths that clear whole clusters of food efficiently instead of zig-zagging to whichever pellet is nearest at each step.

**Once Completed Get a PDF of the completed File and Push into the Week 04 branch in the Github Project**

Finalize and Lock Lab 04 Documentation

**Incomplete: {filledCount}/{fields.length} questions answered. Please provide detailed responses for all questions.**

**FACULTY OF COMPUTING**